Full-Stack Application Development

Databases & SQL Fundamentals

Week 3 — Databases & SQL Fundamentals

Instructor
Dilovan Matini

Topics This Week
Databases, SQL, CRUD

What We Learned So Far

Recap from Previous Weeks

Week 1

- HTML
- CSS
- JavaScript basics
- Browser vs Server

Week 2

- Variables
- Conditions
- Loops
- Functions

Important Question

Where does application data get stored permanently?

Why Applications Need Databases

The Problem Without Databases

Without Databases

- Users disappear after refresh
- Messages are lost
- Login systems fail
- Products cannot be saved
- Orders disappear

Real Examples Using Databases

- Instagram
- Facebook
- TikTok
- Amazon
- Banking Systems

What Is a Database?

Understanding data storage systems

A **database** is a system used to store and organize information.

Examples of Stored Data

- Users
- Passwords
- Products
- Orders
- Messages
- Student records

Popular Databases

- MySQL
- PostgreSQL
- SQLite
- MongoDB

In This Course

MySQL

Stores & Organizes

All Your Information

Database Analogy

Database = Library

Real World	Database
Library	Database
Bookshelf	Table
Book	Record
Book title	Column

Another Analogy

Excel Sheet

≈

Database Table

Key Takeaway

Think of a database as a **digital filing cabinet**. Each drawer is a table, each folder is a row, and each label on the folder is a column. This structure keeps everything organized and easy to find!

Tables, Rows, and Columns

Understanding Database Structure

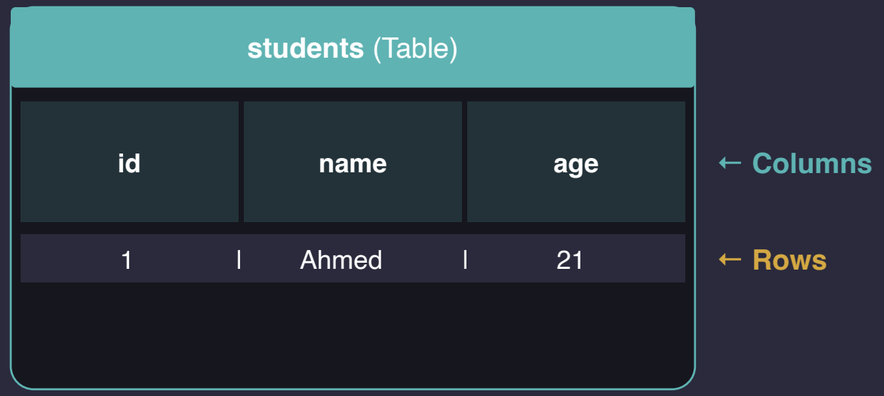
Table Example: students

id	name	age
1	Ahmed	21
2	Sara	19

Important Terms

Concept	Meaning
Table	Collection of data
Row	One record
Column	One field / type

Visual Breakdown



Columns define the structure — what type of data each field holds (id, name, age).

Rows contain the actual data — each row is one student record.

Table = the container that holds all related rows and columns together.

SQL Basics

Structured Query Language

SQL =

Structured Query Language

Used to:

- Create databases
- Create tables
- Insert data
- Read data
- Update data
- Delete data

SQL = Talking to the Database

SQL is the language you use to communicate with your database. You write commands in SQL, and the database understands and executes them.

Just like English lets you talk to people, **SQL lets you talk to databases**.

SQL Basics

Creating databases, tables, and working with data types

Creating a Database

Your first SQL command

```
CREATE DATABASE school_system;
```

Explain

- **CREATE** → make something
- **DATABASE** → database type
- **school_system** → database name

Naming Tips

- Use **lowercase** letters
- No **spaces** in names
- Keep names **simple** and descriptive

Good examples:

- school_system
- online_store
- my_app

Creating Tables

Define the structure of your data

SQL

```
CREATE TABLE students (
id INT,
name VARCHAR(100),
age INT
);
```

Explain

Keyword	Type	Meaning
INT	Integer	Whole numbers
VARCHAR	Text	Variable-length text
100	Limit	Maximum characters

Table Structure

students

id (INT)	**name** (VARCHAR)	**age** (INT)

Each column has a **name** and a **data type** that defines what kind of data it can store.

Database Data Types

Common Data Types

Type	Purpose
INT	Numbers (integers)
VARCHAR	Short text (up to N characters)
TEXT	Long text (unlimited length)
DATE	Dates (YYYY-MM-DD)
BOOLEAN	True / False values

Examples

- Age → **INT**
- Name → **VARCHAR**
- Description → **TEXT**

Choosing Data Types

Pick the **smallest type** that fits your data.

This saves storage and improves performance.

Rule of thumb:

Numbers → INT

Short text → VARCHAR

Long text → TEXT

Dates → DATE

Yes/No → BOOLEAN

Primary Keys

Uniquely identifying each record

What is a Primary Key?

- Uniquely identifies **each row**
- Prevents **duplicate records**
- Usually named: **id**

Example

id	name
1	Ali
2	Sara

IDs should not repeat. Each student gets a unique number.

Think of It Like...

A **primary key** is like a student ID card number.

No two students have the same ID.

This ensures every record can be uniquely found.

Adding Data

INSERT INTO statement

SQL

```sql
INSERT INTO students (id, name, age)
VALUES (1, 'Ali', 22);
```

Explain

- **INSERT INTO** → add data to the table
- **VALUES** → the actual data values

Result in Table

id	name	age
1	Ali	22

Remember

The order of values must match the order of columns!

Correct:

(id, name, age)

(1, 'Ali', 22)

Wrong:

(id, name, age)

(1, 22, 'Ali')

Reading Data

SELECT statement

SQL

```
SELECT * FROM students;
```

Meaning

Keyword	Meaning
SELECT	Get data
*	Everything (all columns)

FROM students → from students table

This command retrieves **all records** from the students table, showing every column for every row.

The * Wildcard

* means "select all columns."

You can also pick specific columns:

```
SELECT name
FROM students;
```

This only returns the **name** column.

Filtering Records

The WHERE clause

SQL

```
SELECT * FROM students

WHERE age > 20;
```

WHERE

Used to **filter** results. Only rows that match the condition are returned.

Real Examples

- Show only **active users**
- Show only **expensive products**
- Show only **students older than 20**

Common Operators

Op	Meaning
=	Equal to
>	Greater than
<	Less than
<>	Not equal (!=)

Hands-On Practice

Practice Session

Student Tasks

1. Create database
2. Create students table
3. Insert 5 students
4. Display all students
5. Filter students by age

Goal

Become comfortable writing SQL.

Take your time and ask questions if you get stuck!

CRUD Operations

Create, Read, Update, Delete — the foundation of all application data operations

CRUD Operations

The four basic data operations

Letter	Meaning	SQL Command
C	Create	INSERT INTO
R	Read	SELECT
U	Update	UPDATE
D	Delete	DELETE

Every Application Uses CRUD

Whether you're building a social media app, an online store, or a banking system — **all data operations boil down to these four actions**. Master CRUD, and you've mastered the foundation of database interaction.

Create Operation

C — CREATE

```
INSERT INTO students (id, name, age)
VALUES (2, 'Sara', 20);
```

Real Examples

- Add new account
- Add product
- Add message
- Add comment
- Add order

Create = Add New

The Create operation adds **new records** to a table.

Think of it like filling out a form — you're entering new information into the system for the first time.

Read Operation

R — READ

```
SELECT * FROM students;
```

Real Examples

- View products
- View messages
- View posts
- View profile
- View orders

Read = View Data

The Read operation **retrieves** data from the database.

This is the most common operation — every time you browse a website, you're reading data.

Update Operation

U — UPDATE

```
UPDATE students
SET age = 25
WHERE id = 1;
```

Important

WHERE is VERY important.

Without WHERE, **all rows** may change!

Always use WHERE with UPDATE!

UPDATE without WHERE affects every row in the table. This is one of the most common and dangerous mistakes beginners make.

Update = Modify

The Update operation **changes existing** data.

You specify which row(s) to change using WHERE, and what to change using SET.

Delete Operation

D — DELETE

```
DELETE FROM students
WHERE id = 1;
```

Warning

Deleting without WHERE:

```
DELETE FROM students;
```

This removes **ALL records**!

Safe Practice

Always use WHERE with DELETE to specify exactly which record(s) to remove. Double-check your WHERE clause before executing!

Delete = Remove

The Delete operation **removes** records from the table.

Like UPDATE, always use WHERE to target specific rows. Without it, everything is gone!

CRUD in Real Applications

How CRUD powers the apps you use daily

Application	CRUD Example
Instagram	Add / edit / delete posts
YouTube	Upload / delete videos
Banking	Create transactions
University System	Add / update students

Key Insight

Every app you use — from social media to banking — is built on these same four operations. Understanding CRUD means understanding how **all modern applications work**.

Common Beginner Mistakes

Avoid these common SQL errors

✕ Missing Quotes

VALUES (1, Ali) ❌

VALUES (1, 'Ali') ✅

Text values must be wrapped in **single quotes**.

✕ Wrong Table Name

SELECT * FROM student; ❌

SELECT * FROM students; ✅

Table names must match **exactly** — check spelling!

✕ Missing Semicolon

SELECT * FROM students ❌

SELECT * FROM students; ✅

Always end SQL statements with a **semicolon**.

Database Thinking & Mini Project

Designing database structures and applying CRUD operations

How Developers Think About Data

Thinking Like a Developer

When building systems, developers ask themselves these key questions:

1 What data should be stored?

Identify all the information the application needs to keep track of — users, products, orders, etc.

2 What tables are needed?

Group related data into tables. Each table should represent one type of entity.

3 What fields are needed?

Decide what columns each table needs and choose the right data types for each.

4 How are tables related?

Understand how different tables connect — which table references which.

Designing a University Database

Example: University System

Possible Tables

- students
- teachers
- courses
- grades

Students Table

id	name	email

Courses Table

id	title	credits

Design Tip:

Each table should represent **one entity**. The students table only stores student information — not their grades or courses.

This keeps your database **organized** and **easy to maintain**.

Designing an Online Store

Example: E-Commerce System

Possible Tables

- users
- products
- orders
- payments

Products Table

id	name	price

E-Commerce Database Design

An online store needs to track:

Users — who is buying

Products — what is being sold

Orders — what was purchased

Payments — how it was paid for

Each table has a clear, single purpose.

Table Relationships

Database Relationships (Simple Intro)

Example

- One user can have **many orders**
- One student can enroll in **many courses**

Simple Visualization

A student enrolls in a course through the **enrollments** table — this connects the two.

The enrollments table stores which student is in which course — linking the two tables together.

Note

We will study relationships **deeper later** in the course.

For now, just understand that tables can be **connected** to each other through special linking tables.

Group Activity

Mini Project Activity

Build a Simple Database Design — Choose one system below:

Library System

School System

Restaurant System

Hospital System

Students Should

- Define **tables** needed for the system
- Define **columns** for each table
- Decide **data types** for each column

SQL Challenge

SQL Practice Challenge

Students should write **all six** of the following SQL commands:

- CREATE DATABASE
- CREATE TABLE
- INSERT
- SELECT
- UPDATE
- DELETE

Goal

Use **all CRUD operations** together in one complete exercise.

What We Learned

Week 3 Wrap-Up

Databases

- Tables
- Rows
- Columns
- SQL basics

Operations

- Create
- Read
- Update
- Delete

Skills

- Writing basic SQL
- Thinking about application data
- Designing simple database structures

Great job this week!

You now understand the foundation of how applications store and manage data. Next week, we'll connect this knowledge to the backend!

Next Week Preview

Backend Fundamentals & APIs

Next week we will learn:

- **Node.js**
- **Express.js**
- **Backend servers**
- **APIs**
- **Requests & responses**

This Week

Data Storage

Next Week

Application Logic